

LLMOrchestrator

LLMOrchestrator

Јединствена контролна раван за све хеадлесс CLI кодирајуће агенте.

Сажетак

LLMOrchestrator је самостални, вишекратно употребљив Go модул за покретање, управљање и комуникацију са хеадлесс CLI агентима (OpenCode, Claude Цоде, Gemini, Junie, Qwen Цоде) путем хибридног протокола цеви и фајлова, са прекидачима кола по агенту, прикључивим избором више провајдера, одвојеном апстракцијом за интернационализацију и гаранцијама против лажних тестова.

Кратак опис

Вишепут употребљив Go модул који пружа јединствени интерфејс за покретање и управљање више LLM-ом покретаних CLI агената путем хибридног протокола цеви и фајлова. Нитно безбедно груписање агената са прекидачима кола и изборним стратегијама рутирања, намерно независно од потрошача, са прикључивим преводиоцем за интернационализацију.

Детаљан опис

LLMOrchestrator представља заједничку инфраструктуру за оркестрирање хеадлесс CLI кодирајућих агената — ону позадинску структуру коју сваки мултиагентски систем тихо захтева, а обично се лоше поново имплементира. Уместо да сваки пројекат поново развија покретање процеса, уоквиривање порука и парсирање резултата за алате попут OpenCode-а, Claude Цоде-а, Gemini CLI, Junie и Qwen Цоде-а, овај модул нуди јединствени интерфејс Agent, нитно безбедан AgentPool и MultiProviderPool који обједињује агенте из више

провајдера иза јединственог фасадног интерфејса. Рутирање је прикључиво преко AgentSelector-а — кружно распоређивање које прескаче провајдере који не испуњавају захтеве, или распоређивање по приоритету са резервним опцијама — тако да начин дистрибуције посла представља политику коју бирате, а не унапред задату претпоставку. Сваки конкретан агент је танак адаптер над заједничким BaseAdapter-ом који управља целим животним циклусом процеса: покретање са подешавањем цеви, уредно заустављање путем SIGTERM-а па SIGKILL-а, поновно покретање и провера активности — све оне заморне, подложне грешкама делове, решене једном за свагда.

Комуникација је намерно хибридна, прилагођена задатку. Цевни транспорт преноси JSON поруке ограничене новим редом, са временским ограничењем за читање по захтеву и ограничењем дужине одговора за брзу интерактивну размену, док фајл-транспорт користи директоријуме за улазне/излазне фајлове и дељене ресурсе по сесији за велике или трајне артефакте који не би требало да бораве у цеви. Отпорност на грешке није накнадна замисао — она је структурна: прекидач кола по агенту се активира након три узастопна неуспеха, са 60-секундним хлађењем пре полуактивне пробе, а позадински монитор здравља пингује агенте како би онај који је пао могао да се опорави без чекања да га саобраћај примети. Преузимање из базена блокира се на променљивом услову уместо да троши процесорско време у бескорисном чекању, а парсер одговора је без стања и безбедан за конкурентно позивање. Модул је строго одвојен — никакве специфичности потрошача нису дозвољене да процуре унутра — а сваки стринг видљив кориснику пролази кроз прикључиви и18n Translator, са NoopTranslator-ом који враћа идентификаторе порука дословно, тако да недостајући превод буде одмах уочљив уместо да се крије.

Зашто смо га направили

Сваки мултиагентски систем мора поуздано да покреће и комуницира са CLI агентима. Поновно решавање покретања, уоквиривања, парсирања и руковања грешкама по пројекту је расипање времена и подложно грешкама. LLMOrchestrator то централизује у један одвојени, виšekратно употребљив модул чија специјализована одговорност га чини поново употребљивим — а та поновна употребљивост се губи у тренутку када било какве специфичности потрошача процуре унутра.

Зашто је ово револуционарно

Претвара “управљање војском хетерогених CLI агената” из прилагођеног инжењерског мука по пројекту у једноставан увоз библиотеке — са решеним и ојачаним груписањем, прекидањем кола, управљањем животним циклусом и замјењивим рутирањем. А пошто анти-блуф тестови проверавају стварни систем од краја до краја уместо да се задовоље са “компајлира се”, добијате апстракцију којој можете заиста веровати да ради под конкурентношћу и отказима, а не ону која само изгледа исправно на дијаграму.

Шта је иновативно

- **Хибридни протокол цев+датотека** — интерактивна брзина (JSON линије преко стдин/стдоут, рокови за читање, ограничења одговора) и трајна размена заснована на датотекама (inbox/outbox/заједнички простор) за велике артефакте, тако да никада не морате да жртвујете латенцију зарад трајности или обрнуто.
- **Мулти-провајдерски базен са замјењивим селекторима** — један фасадни слој преко више CLI провајдера, са рутирањем кружним редом или по приоритету изабраним као политика, а не уграђеним.
- **Прекидач кола по агенту + монитор здравља у позадини** — аутоматска деградација и опоравак (3 неуспеха → 60с отворено коло → полуотворена проба), тако да нестабилан агент буде изолован, а затим тихо враћен у рад без ручне интервенције.
- **Базен без заузетог чекања** — Acquire блокира на sync.Cond док се не ослободи одговарајући, здрав агент или док се контекст не откаже, тако да чекање не троши процесорско време.
- **Строга раздвојеност + анти-блуф и18н** — NoopTranslator враћа идентификаторе порука дословно, тако да недостајући превод није могуће превидети уместо да се тихо остави празан.
- **Сигурност подразумевано** — листа дозвољених бинарних путања значи да нема интерполације љуске и стога нема површине за убацивање команди, подржано заштитом од преласка путања, ограничењем одговора на 1 МиБ против неконтролисаног излаза и маскирањем API кључева у записницима.
- **Анти-блуф оквир за изазове** — стварни циклуси диск/JSON/парсер кроз пет локализација, са упареном мутационом капијом која мора изаћи са ненултим статусом када је функционалност покварена — тест који доказује да стварно може да откаже.

Највећи технички изазови и како смо их решили

- **Поуздан И/О агената.** Комуникација са покренутим CLI процесом је варљиво тешка; решено хибридним транспортом цев+датотека, дефинисаним уговором порука/парсера тако да обе стране договоре формат на жици, и BaseAdapter-ом који централизује цео животни циклус процеса, укључујући елегантан SIGTERM тајмаут који ескалира на SIGKILL као резервну опцију.
- **Конкурентност без заузетог чекања.** Решено помоћу AgentPool-a са mutexом и условном променљивом, где Acquire спава док се не ослободи агент са одговарајућим могућностима, упарено са безстањским парсером без споредних ефеката који је безбедан за позивање из више горутина истовремено.
- **Изолација отказа провајдера.** Решено тако да један лош провајдер не може да повуче остале: прекидачи кола по агенту ограничавају радијус експлозије, а горутина монитора здравља покреће опоравак чак и када нема захтева који би га активирали.
- **Доказивање исправности, а не само компајлирања.** Решено помоћу покретача изазова: десетине инваријанти на ен/ср/ја/ес/де које тестирају стварни систем, плус упарена мутациона капија (LLMORCH_MUTATE_RUNNER=1 мора да откаже → омотач излази са 99) која намерно квари функционалност како би доказала да сама капија није блуф.
- **Локализација без тихог отказа.** Решено шавом NoopTranslator-a са дословним идентификаторима и убризгавањем преводиоца по потрошачу, тако да празнина у преводима увек буде видљива уместо да се прикрије.

Технолошки стек

- **Go (1.25)** — изабран због врхунске конкурентности и чисте контроле процеса, што је управо оно што захтева оркестрација процеса живих агената; имплементира модул, његове адаптере за агенте, транспортне механизме и парсер.
- **Go стдlib (уз testify, yaml.v3)** — намерни избор да се површина зависности сведе на минимум и да се не увлаче LLM SDK-ови, како би модул остао лаган и могао да се угради у било ког корисника без преношења додатног терета.
- **Транспорт путем цеви (JSON-линес преко стдио)** — изабран за брзу интерактивну размену порука, ојачан временским ограничењима за читање и ограничењима

дужине одговора како би се спречило да заблокиран или неконтролисан агент блокира позиваоца.

- **Транспорт путем фајлова (inbox/outbox/заједнички)** — изабран за трајнију размену великих артефаката по сесији, где би цев била погрешан алат.
- **sync.Mutex/sync.Cond** — изабрани за имплементацију блокирајућег, правичног преузимања агената из базена без активног чекања.
- **Прекидач струјног кола + Надгледник здравља** — изабрани заједно како би обезбедили отпорност по агенту и активан опоравак, а не само детекцију грешака.
- **pkg/i18n Преводилац** — изабран као одвојени слој за локализацију који држи специфичне стрингове корисника изван језгра.
- **Оквир за изазове (challenges/runner) + Макефиле (test-race, fuzz, cover)** — изабран за проверу засновану на доказима, укључујући детекцију трка и фуззинг парсера како би се исправност доказала у неповољним условима, а не само претпоставила.

Статус и напомене о искрености

- **Статус: бета.** Одвојени модул за вишекратну употребу, који се користи као подмодул у више Helix/васиц пројеката. **Лиценца: Апацхе-2.0**; репозиторијум GitHub је јавно доступан.
- Метаподаци модела долазе из LLMsVerifier преко HelixQA; овај модул не увози LLMsVerifier/VisionEngine/DocProcessor директно. Стекови наведени у CLAUDE.md матичне апликације (Gin/PostgreSQL итд.) описују helix_code, а не овај модул.

Приоритетни ниво: Helix-примарни (LLM-инфраструктурни кластер — одвојени модул за вишекратну употребу). Рангира се иза HelixTrack.